Web Development Internship

LOOP

AI CUSTOMER FEEDBACK
INTELLICENCE PLATFORM

Team Members

1.Goutam Kushwah

2.Gaurav Athode

3.Sneha Sekar

4.Mohan Sahu

Project Repository

https://github.com/goutamkushwah/Project-loop

Live Application

https://zidio-loop.vercel.app

Acknowledgement

We would like to acknowledge **Zidio Development** for providing the Web Development Track internship under which LOOP was developed.

The project created an opportunity to work across several areas of modern full-stack software development, including application architecture, authentication, role-based authorization, relational database design, multi-tenancy, AI integration, semantic retrieval, data visualization, application security, deployment, and quality verification.

We also acknowledge the documentation and development ecosystems surrounding Next.js, React, TypeScript, PostgreSQL, Prisma, NextAuth.js, Zod, Google Gemini, pgvector, Recharts, Tailwind CSS, and Vercel, which supported the implementation of the project.

Specific mentor names or individual external contributors were **not specified in the available project information** and therefore are not listed here.

Table of Contents

Abstract

Customer feedback is an important source of product knowledge, but in practical environments it is often distributed across multiple channels such as support tickets, reviews, surveys, customer-success notes, and other forms of customer communication. When the volume of feedback increases, manually reading, tagging, categorizing, comparing, and summarizing every item becomes difficult and inconsistent.

LOOP — AI Customer Feedback Intelligence Platform was developed to address this problem through a multi-tenant software platform that centralizes customer feedback and transforms it into structured and evidence-backed product intelligence.

The platform supports manual feedback entry, CSV bulk import, simulated channel ingestion, a searchable feedback inbox, filtering, pagination, workflow management, dashboard analytics, AI-assisted sentiment and theme classification, theme intelligence, trend analysis, semantic embeddings, retrieval-grounded natural-language question answering, and Voice-of-Customer reporting.

Google Gemini is integrated on the server side for AI-related processing. Generated classification output is treated as untrusted application input until it has been parsed and validated. Zod schemas are used to validate structured AI responses before persistence.

Ask LOOP uses a Retrieval-Augmented Generation architecture. A user question is converted into an embedding, relevant feedback is retrieved from PostgreSQL using pgvector, and only the retrieved workspace evidence is provided to Gemini for answer generation. Supporting feedback references are returned so users can inspect the evidence behind the response.

Voice-of-Customer reports follow a different but related reliability principle. AI narrative is generated.The application is implemented using Next.js 14, React 18, TypeScript, Tailwind CSS, PostgreSQL, Prisma 5, NextAuth.js 4, Zod, Google Gemini, pgvector, HNSW indexing, Recharts, and Vercel.

The result is an internship-scale but production-style implementation demonstrating full-stack development, multi-tenancy, AI integration, retrieval, database design, security, analytics, deployment, and software quality practices.

Introduction

1.1 Overview

Customer feedback can directly influence product strategy, feature prioritization, usability improvements, and customer satisfaction. However, feedback only creates value when teams can organize and interpret it effectively.

A product team may receive hundreds or thousands of comments across multiple sources. Some describe bugs, some request features, some express satisfaction, and others highlight usability or business concerns. The challenge is not only collecting these messages but turning them into reliable insights.

LOOP was developed as a centralized feedback-intelligence application designed around the following workflow:

Ingestion

↓

Classification

↓

Analysis

↓

Retrieval

↓

Reporting

↓

Action

The system combines conventional software-engineering components such as authentication, RBAC, relational storage, filtering, pagination, and analytics with AI-assisted classification, semantic retrieval, grounded question answering, and report narrative generation.

1.2 Purpose of the Project

The purpose of LOOP is to demonstrate how AI can be integrated into a full-stack SaaS-style application without allowing the language model to become the sole source of application truth.

The system therefore makes an important distinction between:

- deterministic application logic;
- persisted database information;
- retrieved customer evidence; and
- generated AI content.

For example, feedback counts are calculated by application logic, while Gemini assists with interpreting and summarizing text.

1.3 Project Context

LOOP was developed for the:

Zidio Development — Web Development Track

The project demonstrates skills across:

- frontend development;
- backend development;
- database design;
- authentication;
- authorization;
- multi-tenancy;
- AI integration;
- data visualization;
- deployment; and

Background and Motivation

2.1 Customer Feedback as Product Data

Customer feedback contains qualitative information that cannot always be represented by conventional analytics.

Metrics can indicate that a conversion rate decreased, but customer comments may explain why.

2.2 Fragmentation of Feedback

Feedback is often distributed across:

- support systems;
- review platforms;
- sales conversations;
- community channels.

This fragmentation makes cross-channel analysis difficult.

2.3 AI as an Assistance Layer

Large language models are useful for working with unstructured text, but a production application cannot safely treat generated text as automatically correct.

This motivated several design decisions in LOOP:

1. structured classification output is validated;
2. model-generated classification is persisted;
3. Ask LOOP retrieves evidence before generation;
4. report statistics are calculated before narrative generation;
5. API keys stay on the server.

These decisions make AI a controlled component of the software workflow rather than an unrestricted source of application state.

Problem Statement

The project addresses five main problems.

3.1 Scattered Feedback

Feedback originating from different channels is difficult to search and analyze as a single dataset.

3.2 Manual Analysis

Manually reading and tagging all incoming feedback requires significant effort and becomes less practical as data volume increases.

3.3 Inconsistent Categorization

Different people may classify similar comments differently, making theme-level analysis inconsistent.

3.4 Weak Trend Visibility

A basic feedback list does not easily reveal whether a theme is increasing, decreasing, or newly emerging.

3.5 Evidence Gaps

A summary without traceable supporting feedback can make it difficult for a product team to verify why a conclusion was reached.

LOOP addresses these issues by centralizing the data and connecting analysis back to the original evidence.

Objectives

The objectives of LOOP are:

1. Build a multi-tenant SaaS-style application.
2. Implement authentication and protected application routes.
3. Support ADMIN, ANALYST, and VIEWER roles.
4. Enforce role authorization on the server.
5. Isolate workspace data between tenants.
6. Support manual feedback entry.
7. Support CSV bulk import.
8. Support simulated feedback channels.
9. Build a searchable feedback inbox.
10. Implement server-side pagination.
11. Support compound feedback filtering.
12. Support the feedback triage workflow.
13. Provide filter-aware dashboard analytics.
14. Automatically classify feedback with Google Gemini.
15. Validate structured AI output using Zod.
16. Persist classification results.
17. Create theme assignments and counts.
18. Provide theme drill-down to supporting feedback.
19. Analyze theme trends over time.
20. Compare current and previous periods.
21. Detect emerging/spiking themes.
22. Generate semantic embeddings.

23. Store vectors in PostgreSQL using pgvector.
24. Perform semantic retrieval.
25. Implement retrieval-grounded Ask LOOP.
26. Return supporting evidence with AI answers.
27. Generate Voice-of-Customer reports.
28. Calculate report statistics deterministically.
29. Save generated reports.
30. Support secure public report sharing.
31. Deploy the project through Vercel.
32. Provide quality and deployment verification workflows.

Scope of the Project

5.1 Implemented Scope

The implemented project includes:

- multi-tenant workspaces;
- workspace isolation;
- authentication;
- credentials login;
- RBAC;
- Admin, Analyst, and Viewer roles;
- manual ingestion;
- CSV ingestion;
- simulated channels;
- inbox search;
- filtering;
- pagination;
- sentiment analysis;
- sentiment scoring;
- feature-area labels;
- themes;
- confidence values;
- theme catalog;
- theme drill-down;
- trend analysis;

- spike detection;
- Gemini embeddings;
- pgvector storage;
- HNSW vector search;
- Ask LOOP;
- retrieval-grounded generation;
- evidence citations;
- Voice-of-Customer reports;
- saved reports;
- report sharing;
- security controls;
- Vercel deployment.

5.2 Intentionally Excluded Scope

The current implementation does not include:

- billing or payment processing;
- native mobile applications;
- real-time collaborative editing using WebSockets;
- email/SMS delivery infrastructure;
- live third-party feedback integrations.

Simulated feedback channels are intentionally used to demonstrate integration-style ingestion without requiring external provider accounts.

5.3 Future Scope

Possible future extensions are described separately in Chapter 39 and must not be confused with currently implemented functionality.

Existing System

6.1 Conventional Feedback Workflow

The project does not provide details of a specific pre-existing software system being replaced.

Therefore:

> **A specific existing commercial or organizational system is not specified in the available project information.**

The practical problem can instead be compared with a conventional manual workflow where:

1. feedback exists across several sources;
2. team members read messages individually;
3. tags are added manually;
4. themes are compiled manually;
5. reports require manual summarization;
6. searching by meaning is difficult;
7. evidence can become disconnected from conclusions.

6.2 Limitations of the Conventional Approach

Such workflows may lead to:

- duplicated effort;
- inconsistent labels;
- missed patterns;
- difficulty comparing time periods;
- dependence on individuals remembering historical feedback.

LOOP was designed to provide one integrated workflow for these activities.

Proposed System

7.1 Proposed Solution

LOOP provides a centralized SaaS-style application where customer feedback is ingested, stored, analyzed, searched, and summarized.

flowchart LR

A[Customer Feedback] --> B[Ingestion]

B --> C[Centralized Inbox]

C --> D[AI Classification]

D --> E[Themes]

E --> F[Trend Analysis]

F --> G[Ask LOOP]

G --> H[VoC Reporting]

H --> I[Evidence-backed Decisions]

7.2 Key Design Principle

The system does not assume that AI-generated output is automatically trustworthy.

Instead:

- generated classification is validated;
- retrieval is performed before Ask LOOP generation;
- report statistics are calculated before narrative generation;
- evidence IDs are checked before presentation.

7.3 Result

The proposed system creates a complete path from individual customer comments to broader product intelligence while maintaining traceability.

System Requirements

8.1 Software Requirements

Requirement	Specification
Runtime	Node.js 20 or newer
Package Manager	npm
Source Control	Git
Application Framework	Next.js 14
Database	PostgreSQL
Vector Extension	pgvector
ORM	Prisma 5
AI Access	Google Gemini API key
Deployment	Vercel
Browser	Modern browser capable of running the deployed Next.js interface

8.2 Development Dependencies

Important development tools include:

- TypeScript;
- ESLint;
- Prettier;

8.3 Hardware Requirements

Specific minimum hardware specifications were:

Not specified in the available project information.

The project is primarily a web application and therefore depends on the requirements of Node.js, the selected PostgreSQL host, and the development/browser environment.

Functional Requirements

ID	Functional Requirement
FR-01	Users shall be able to authenticate using credentials.
FR-02	Protected routes shall require an authenticated session.
FR-03	Users shall belong to a workspace.
FR-04	Workspace data shall be isolated by tenant.
FR-05	The application shall support Admin, Analyst, and Viewer roles.
FR-06	Authorization shall be enforced server-side.
FR-07	Authorized users shall be able to create feedback manually.
FR-08	Authorized users shall be able to import CSV feedback.
FR-09	Authorized users shall be able to ingest simulated-channel feedback.
FR-10	Users shall be able to browse feedback using server-side pagination.
FR-11	Users shall be able to search feedback content.
FR-12	Users shall be able to filter feedback by supported dimensions.
FR-13	Authorized users shall be able to update triage status.
FR-14	Feedback shall support Gemini classification.
FR-15	AI classification shall be validated before persistence.
FR-16	Feedback may be associated with one or more themes.
FR-17	Users shall be able to inspect theme counts and supporting feedback.
FR-18	The system shall provide theme-volume trends.
FR-19	The system shall compare current and previous periods.
FR-20	The system shall identify configured spike/emerging-theme conditions.
FR-21	Feedback shall support semantic embeddings.
FR-22	Ask LOOP shall retrieve relevant workspace feedback before answering.
FR-23	Ask LOOP answers shall expose supporting evidence.
FR-24	Authorized users shall be able to generate VoC reports.
FR-25	Reports shall be persisted.
FR-26	Reports shall support secure read-only sharing.

ID	Functional Requirement
FR-27	The dashboard shall display persisted feedback analytics.

Non-Functional Requirements

Requirement	Implementation Consideration
Security	Authentication, RBAC, tenant scoping, input validation and secret isolation
Data Isolation	Workspace-scoped queries for tenant-owned data
Reliability	Structured AI validation and deterministic calculations where possible
Maintainability	TypeScript, service-layer separation, Prisma and reusable validation
Scalability	Server-side pagination and indexed database/vector retrieval
Traceability	Evidence links between AI insights and source feedback
Responsiveness	Responsive application UI
Accessibility	Error/loading handling, keyboard navigation and reduced-motion considerations were included during project hardening
Deployability	Environment-driven configuration and Vercel deployment
Observability/ QA	Health endpoint, build validation, seed verification and smoke testing

Exact performance, latency, throughput, uptime, and load-test requirements were:

Not specified in the available project information.

Technology Stack

Table 1 — Technology Stack

Layer	Technology	Usage
Framework	Next.js 14 App Router	Full-stack pages and server routes
UI Library	React 18	Interactive user interface
Language	TypeScript	Typed frontend/backend development
Styling	Tailwind CSS	Responsive UI styling
Database	PostgreSQL	Relational application storage
ORM	Prisma 5	Typed database access and migrations
Authentication	NextAuth.js 4	Credentials authentication and sessions
Validation	Zod	API and AI-output validation
AI	Google Gemini via @google/genai	Classification, generation and embeddings
Embeddings	Gemini Embeddings	Semantic vector generation
Vector Store	pgvector	Vector storage and similarity search
Vector Index	HNSW	Efficient nearest-neighbor search
Visualization	Recharts	Charts and analytics
Deployment	Vercel	Production application hosting

11.1 Why Next.js

Next.js allows LOOP to combine server-rendered application pages and server-side Route Handlers within one TypeScript codebase.

11.2 Why PostgreSQL

The project requires relational consistency for workspaces, users, feedback, themes, and reports while also supporting vector search through pgvector.

11.3 Why Prisma

Prisma provides schema definition, migrations, typed application access, and a structured way to manage relational data.

11.4 Why Zod

Zod validates external input at runtime.

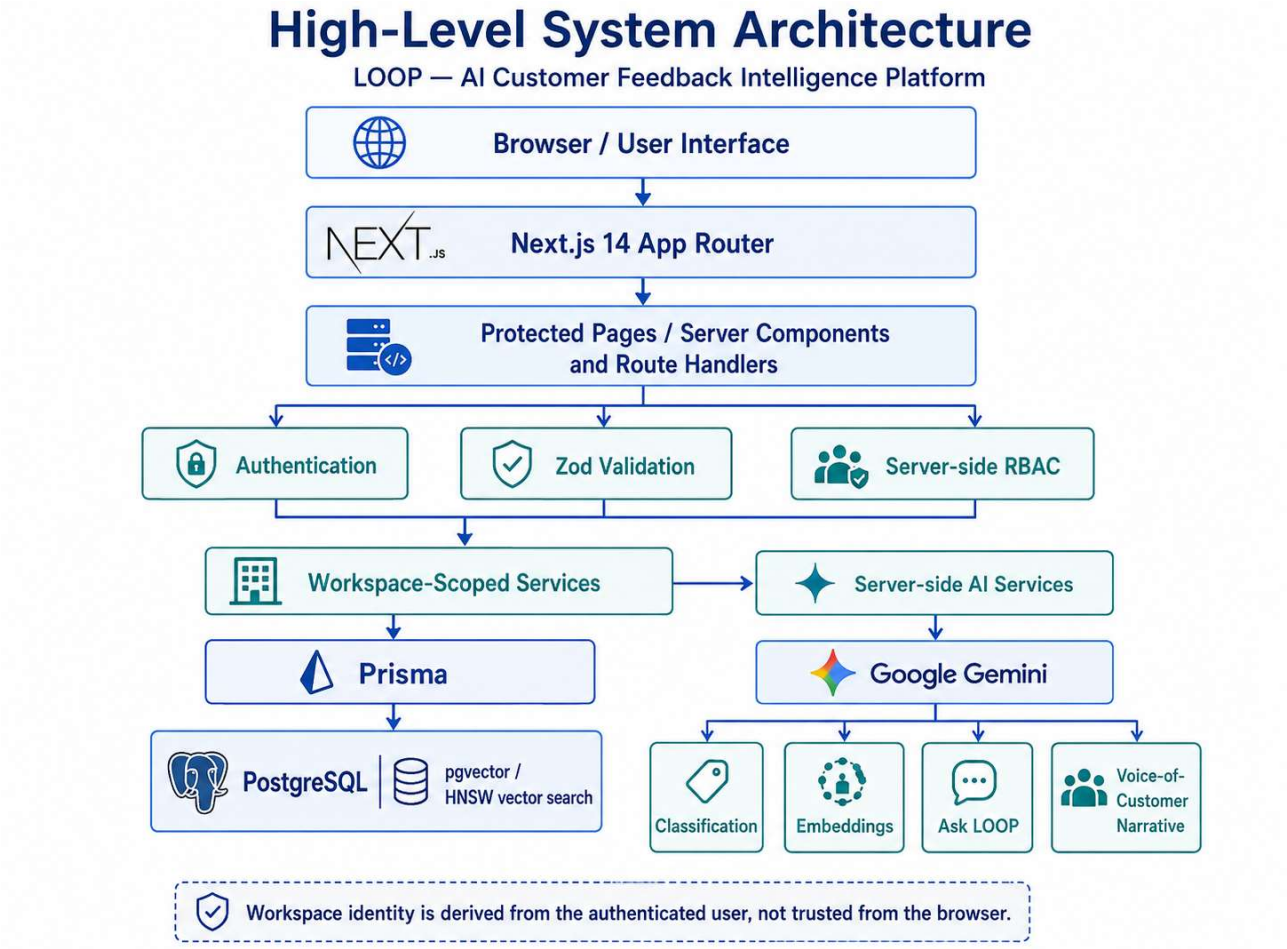
This is particularly important because TypeScript types disappear at runtime and cannot guarantee that:

- browser input;
- CSV input; or
- AI-generated JSON

matches the expected application schema.

System Architecture

12.1 High-Level Architecture



12.2 Architectural Layers

Presentation Layer

Contains application pages, forms, tables, charts, and user interactions.

Route/API Layer

Responsible for:

- authentication checks;
- authorization checks;

- request validation;
- response handling.

Service Layer

Contains workspace-scoped business logic.

Examples include:

- feedback operations;
- theme analysis;
- report generation.

Persistence Layer

Prisma communicates with PostgreSQL.

AI Layer

Server-only Gemini services perform:

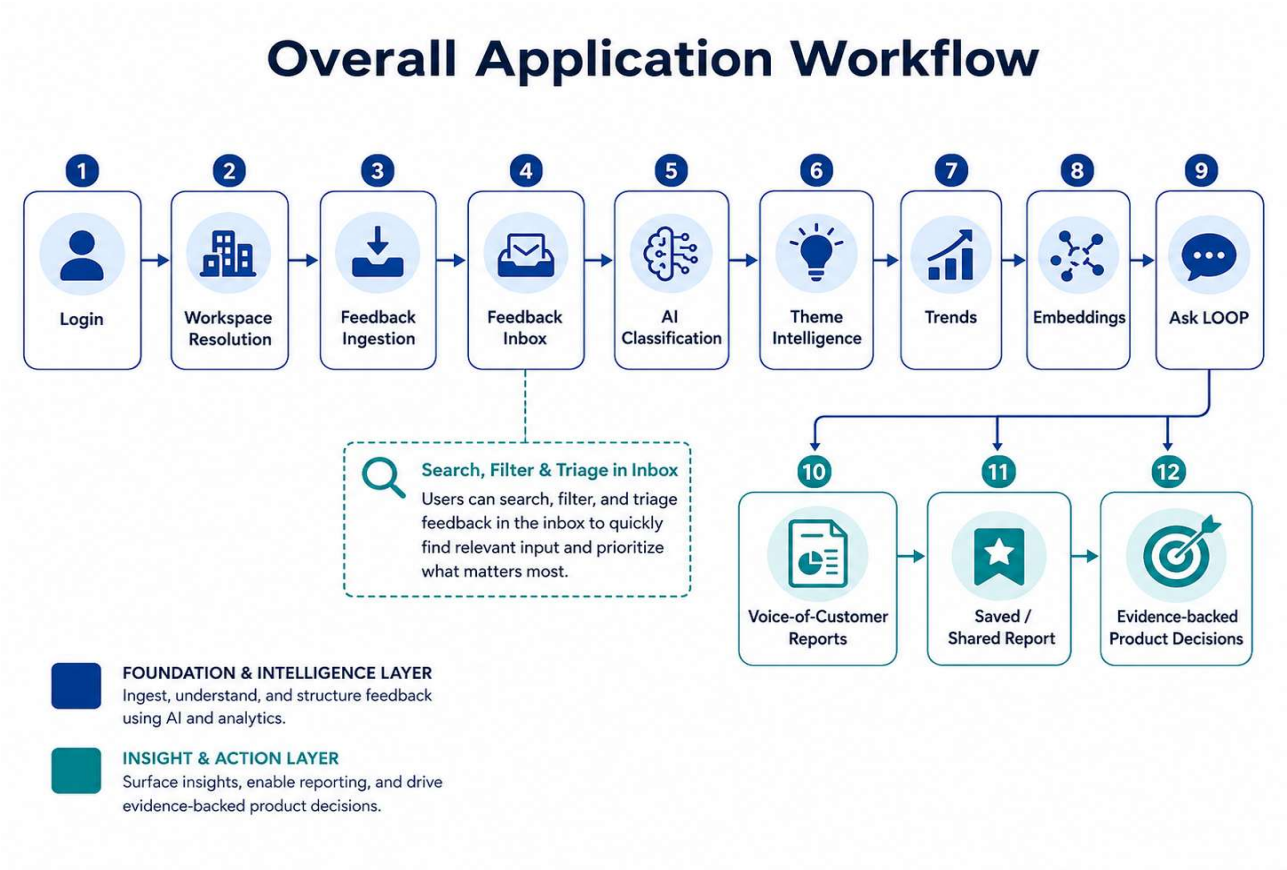
- structured classification;
- embeddings;

12.3 Security Decision

The browser is not trusted to specify the authoritative tenant.

Workspace identity is derived from the authenticated user.

Application Workflow

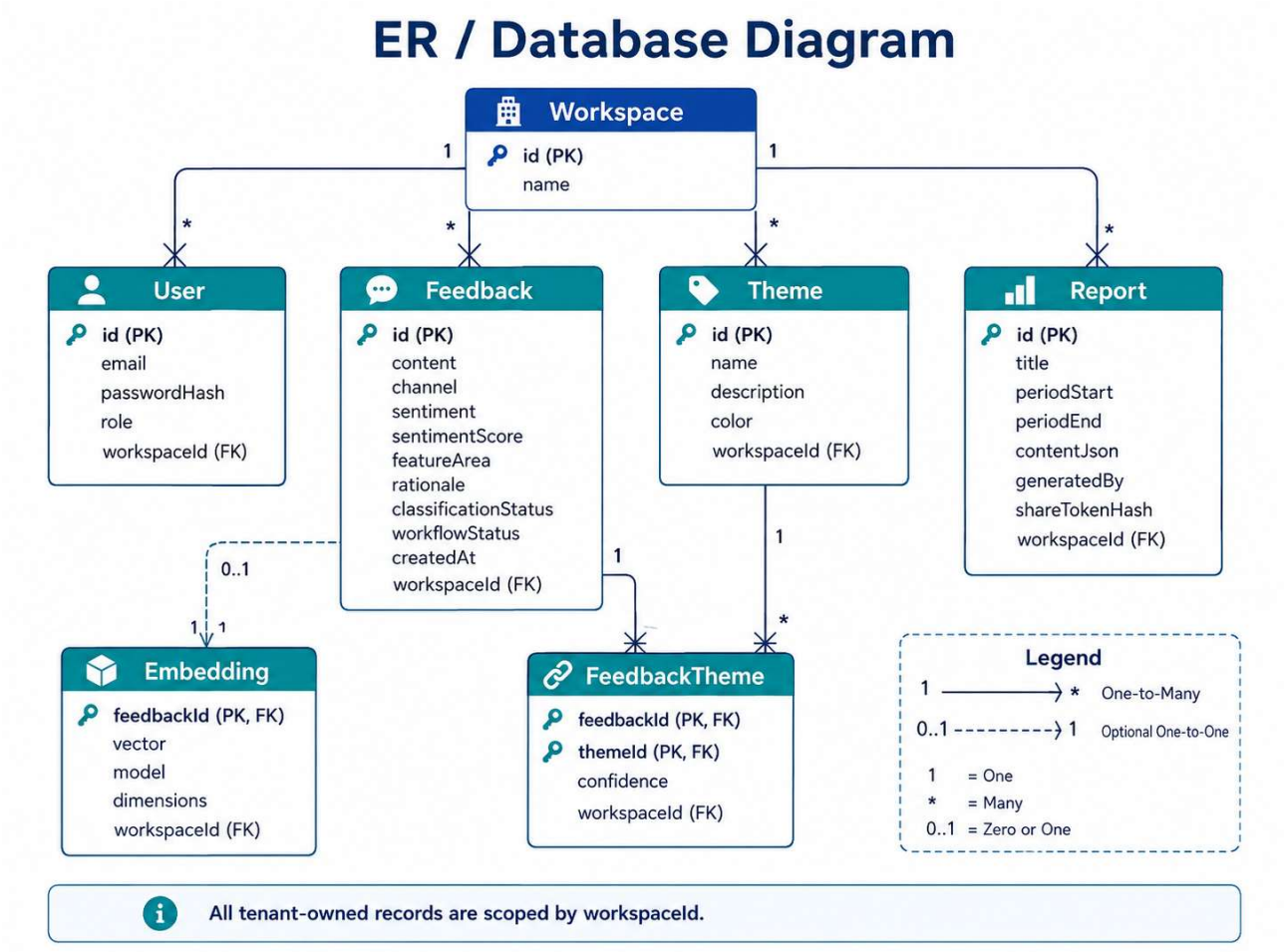


13.1 Typical User Journey

1. User logs in.
2. The backend resolves the user's workspace.
3. User opens the dashboard or inbox.
4. Feedback is created/imported.
5. Authorized feedback is classified.
6. Users search/filter or triage feedback.
7. Product managers inspect theme trends.
8. Users ask questions using Ask LOOP.
9. Analysts/Admins generate VoC reports.
10. Reports can be saved and shared.

Database Design

14.1 Main Entities



14.2 Workspace

Attribute	Purpose
Identifier	Uniquely identifies the workspace
Name / workspace metadata	Represents tenant information
Relationships	Owns users, feedback, themes and reports

Purpose

Workspace is the tenant boundary.

All tenant-owned business information belongs to a workspace.

14.3 User

Attribute	Purpose
Identifier	Unique user
Email	Login identity
Password hash	Credential storage
Role	ADMIN, ANALYST or VIEWER
Workspace reference	Tenant ownership

Security

Passwords are stored using scrypt hashes rather than plaintext.

14.4 Feedback

Attribute	Purpose
Content	Customer feedback text
Channel	Feedback source type
Source reference	Optional source metadata
Customer label	Optional identifying label for demo/workflow use
Sentiment	Classified sentiment
Sentiment score	Numeric sentiment value
Feature area	Short AI-generated feature category
Rationale	Classification explanation
Classification status	Processing state
Workflow status	NEW, REVIEWED, ACTIONED
Created date	Used for chronology and trend analysis
Workspace reference	Tenant ownership

Feedback is the main operational entity of LOOP.

14.5 Theme

Attribute	Purpose
Identifier	Theme ID
Name	Human-readable topic
Description	Theme description
Display color	UI representation
Workspace reference	Tenant ownership

Themes represent recurring categories detected across feedback.

14.6 FeedbackTheme

Attribute	Purpose
Feedback ID	Source feedback
Theme ID	Assigned theme
Confidence	Assignment confidence
Workspace ID	Tenant-scoped relationship

This join entity provides a many-to-many relationship.

One feedback item may belong to more than one theme, and one theme may contain many feedback items.

14.7 Embedding

Attribute	Purpose
Feedback ID	Feedback being represented
Vector	Semantic embedding
Provider/model metadata	Tracks embedding configuration
Dimension metadata	Ensures compatible vectors
Workspace ID	Tenant ownership

The project implementation uses **768-dimensional vectors** for semantic retrieval.

Vectors are stored using PostgreSQL pgvector.

14.8 Report

Attribute	Purpose
Identifier	Report ID
Title	Report title
Period start	Beginning of report interval
Period end	End of report interval
Content JSON	Persisted structured report
Generated-by reference	User that generated it
Workspace reference	Tenant ownership
Share state	Public sharing configuration
Share-token hash	SHA-256 hash of capability token

Reports preserve a snapshot of generated VoC intelligence.

Authentication and Authorization

15.1 Authentication

LOOP uses NextAuth.js 4 with a Credentials provider.

The authentication system handles:

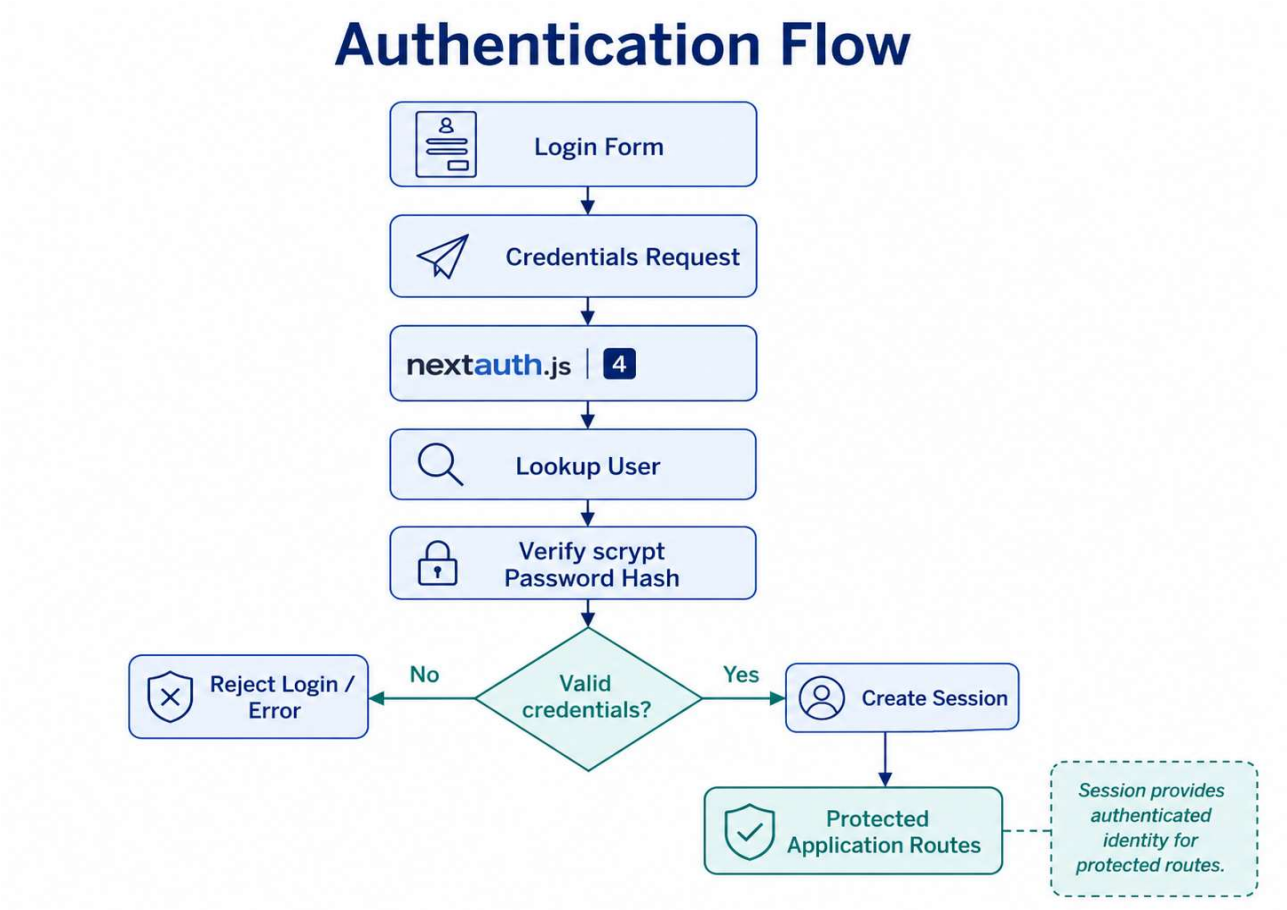
- login;
- protected routes;

15.2 Password Handling

Passwords are hashed using scrypt.

Plaintext passwords are not used as the persistent database representation.

15.3 Authentication Flow

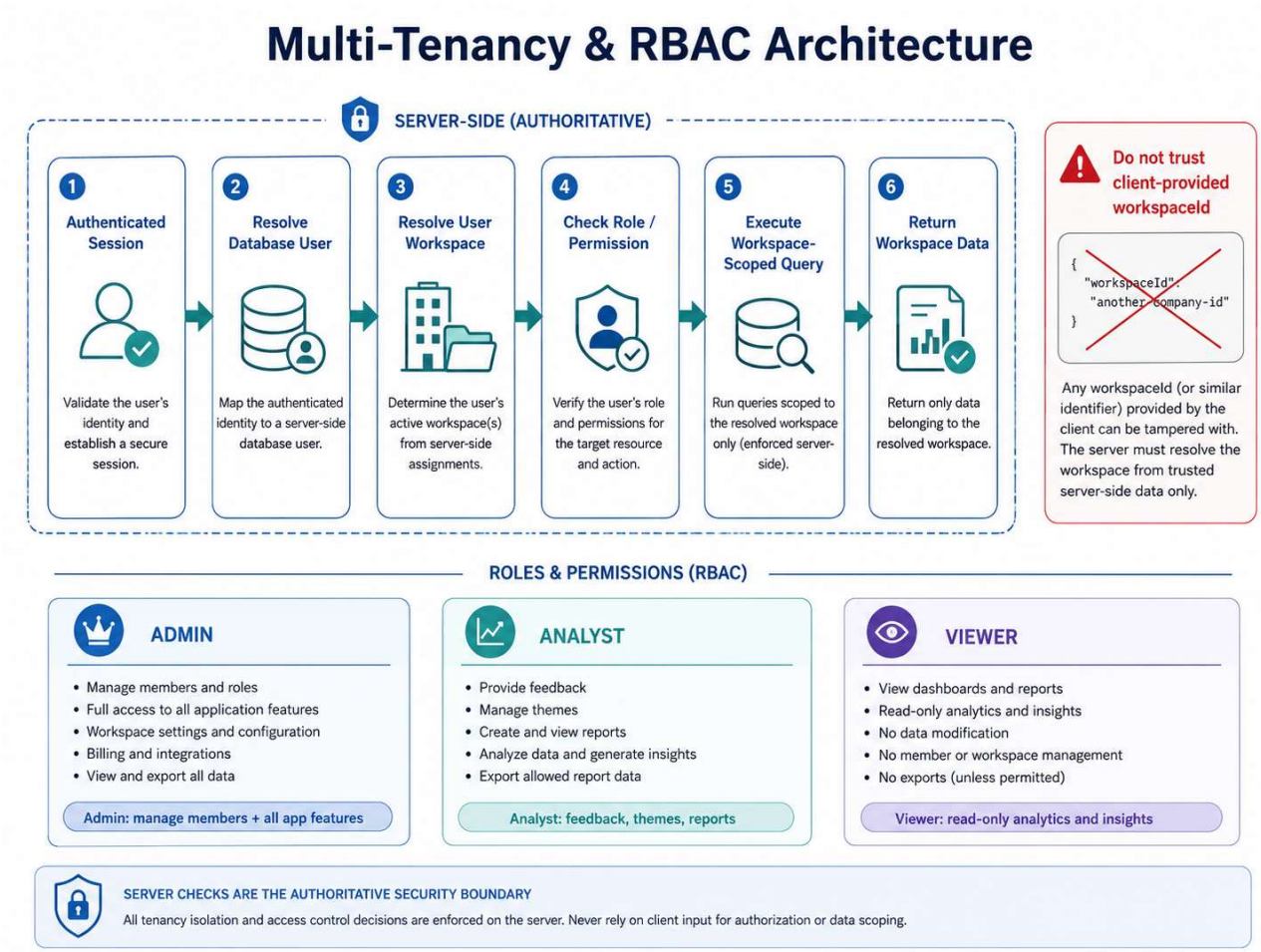


Multi-Tenancy and RBAC

16.1 Multi-Tenant Design

LOOP supports multiple isolated workspaces.

Each workspace represents a tenant.



16.2 Roles

Admin

Can access application intelligence features and manage workspace members and roles.

Analyst

Can manage feedback and use intelligence features but cannot manage workspace membership.

Viewer

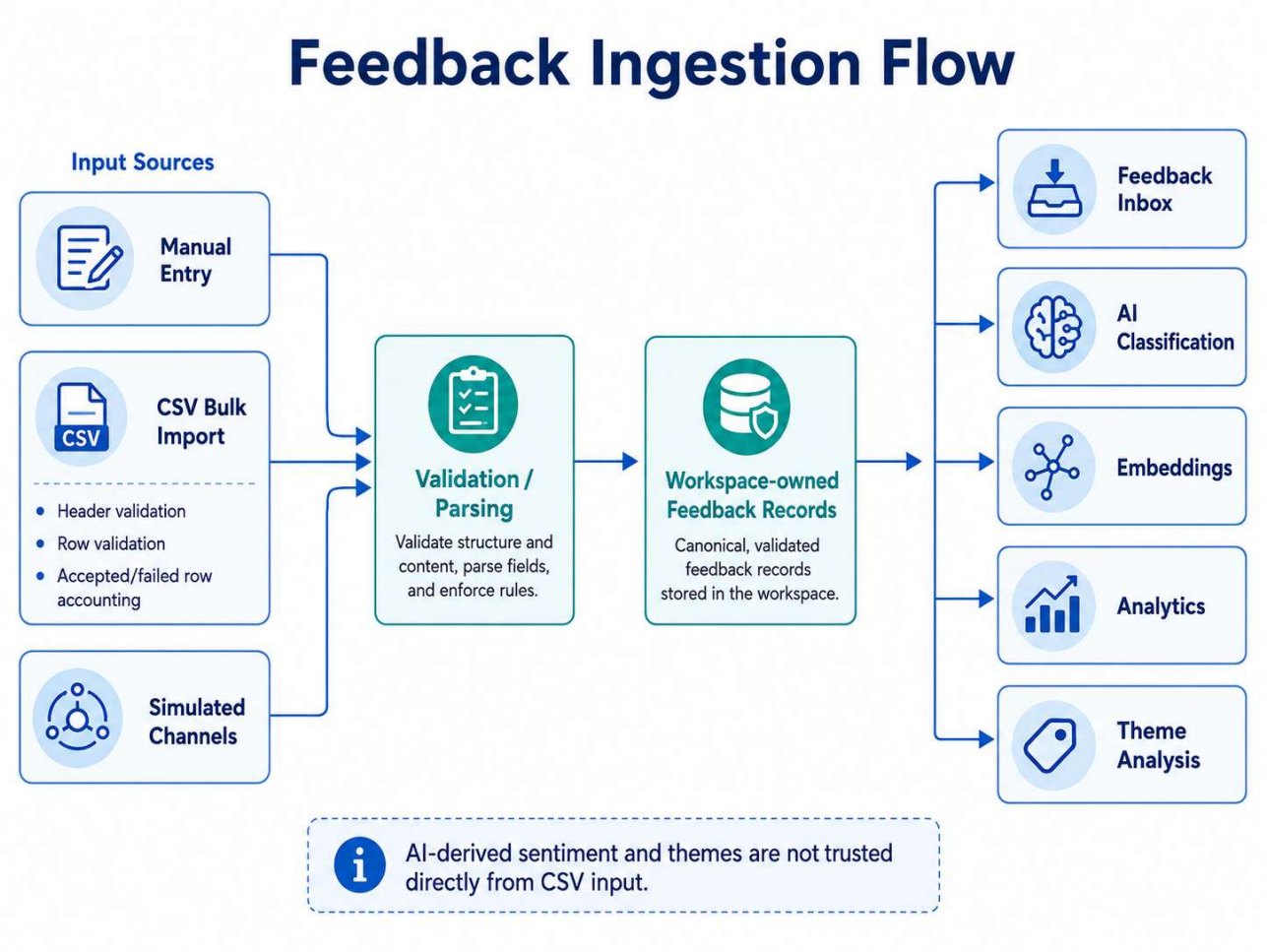
Can inspect read-only analytics and intelligence but cannot perform feedback-management or administration mutations.

16.3 RBAC Permission Matrix

Capability	ADMIN	ANALYST	VIEWER
View dashboard	☑	☑	☑
View inbox	☑	☑	☑
Search/filter feedback	☑	☑	☑
View themes	☑	☑	☑
View trends	☑	☑	☑
Use Ask LOOP	☑	☑	☑
Read reports	☑	☑	☑
Create feedback	☑	☑	✗
CSV import	☑	☑	✗
Simulated ingestion	☑	☑	✗
Update feedback status	☑	☑	✗
Re-classify feedback	☑	☑	✗
Run theme clustering	☑	☑	✗
Generate reports	☑	☑	✗
Share/revoke reports	☑	☑	✗
Manage members/roles	☑	✗	✗

UI controls may reflect permissions, but server checks are the authoritative security control.

Feedback Ingestion



17.1 Purpose

Feedback ingestion brings customer comments into a normalized workspace dataset.

17.2 Manual Entry

Authorized users can enter individual feedback items.

The workflow validates required data before persistence.

17.3 CSV Bulk Import

CSV import allows multiple records to be added efficiently.

Implemented behavior includes:

- file parsing;
- header validation;
- row validation;
- accepted/failed-row accounting;
- bounded processing;
- tenant ownership.

A feedback CSV template is part of the project.

Typical columns include:

content,channel,customer_label,created_at

AI-derived sentiment and theme values are not trusted directly from CSV content.

17.4 Simulated Channels

The project provides simulated channel sources.

These generate realistic feedback items without requiring live external integrations.

17.5 Processing

After ingestion, feedback becomes available for:

- classification;
- embeddings;
- inbox browsing;
- analytics;
- theme analysis.

Feedback Management and Inbox

18.1 Purpose

The inbox is the operational area for searching, filtering, reviewing, and actioning customer feedback.

18.2 Features

The inbox supports:

- server-side pagination;
- full-text search;
- channel filter;
- sentiment filter;
- theme filter;
- status filter;
- date-range filter;
- combined filters;
- workflow updates.
-

18.3 Why Server-Side Pagination

Loading the complete dataset into the browser becomes inefficient as data grows.

Server-side pagination allows the database to return only the requested page.

18.4 Search

Feedback content is searchable using database-backed full-text capabilities implemented in the project.

AI Classification

19.1 Purpose

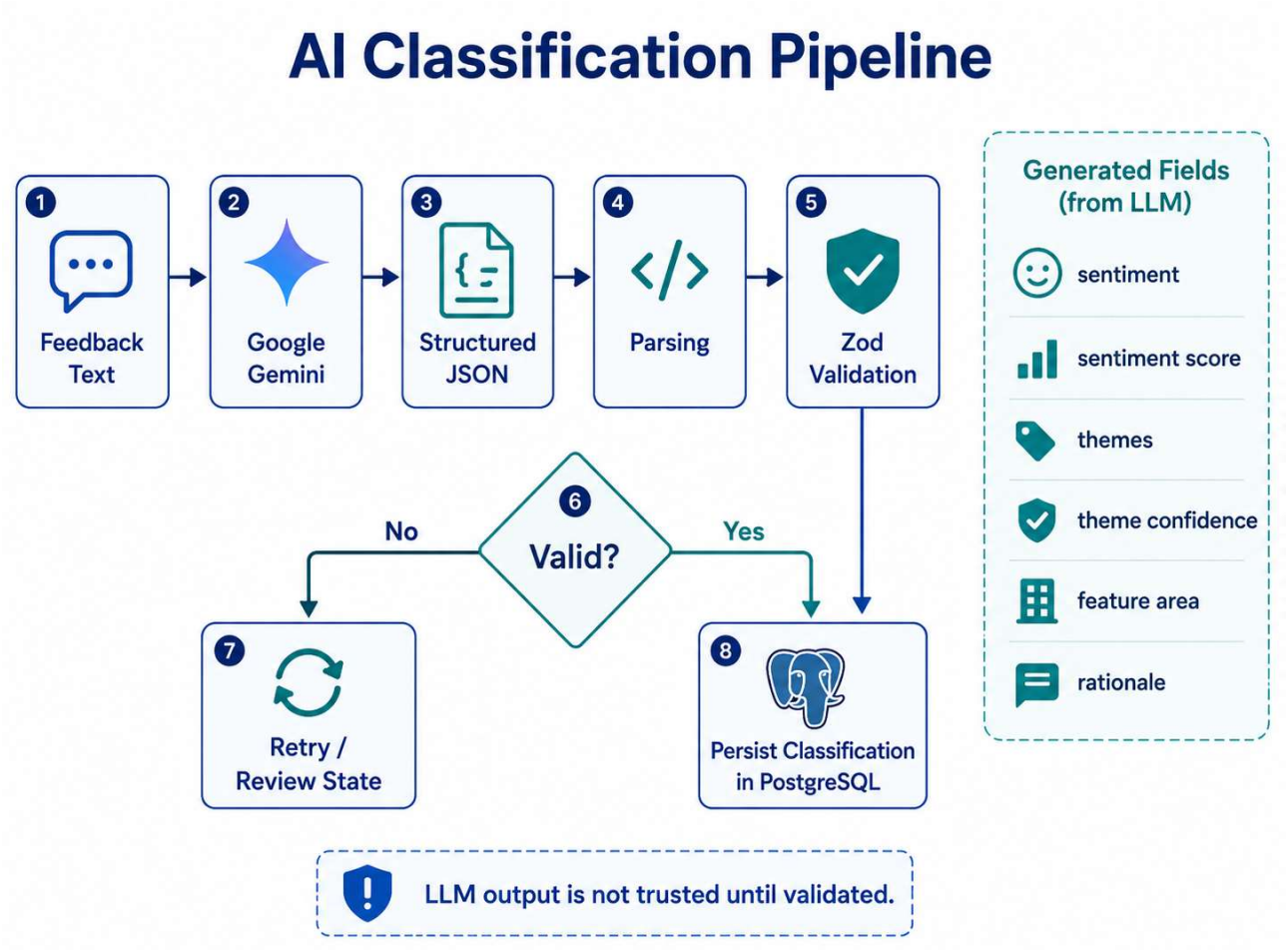
The classification pipeline converts unstructured feedback text into structured metadata useful for filtering and analysis.

19.2 Generated Fields

Classification includes:

- sentiment;
- themes;
- rationale.

19.3 Pipeline



19.4 Why Validation Is Required

LLM-generated text is probabilistic.

TypeScript cannot guarantee runtime correctness because the model response originates outside the trusted application.

A response could contain:

- invalid JSON;
- missing properties;
- invalid sentiment labels;
- incorrect numeric ranges;
- malformed theme structures.

Therefore LOOP validates generated structures using Zod before persistence.

19.5 Persistence

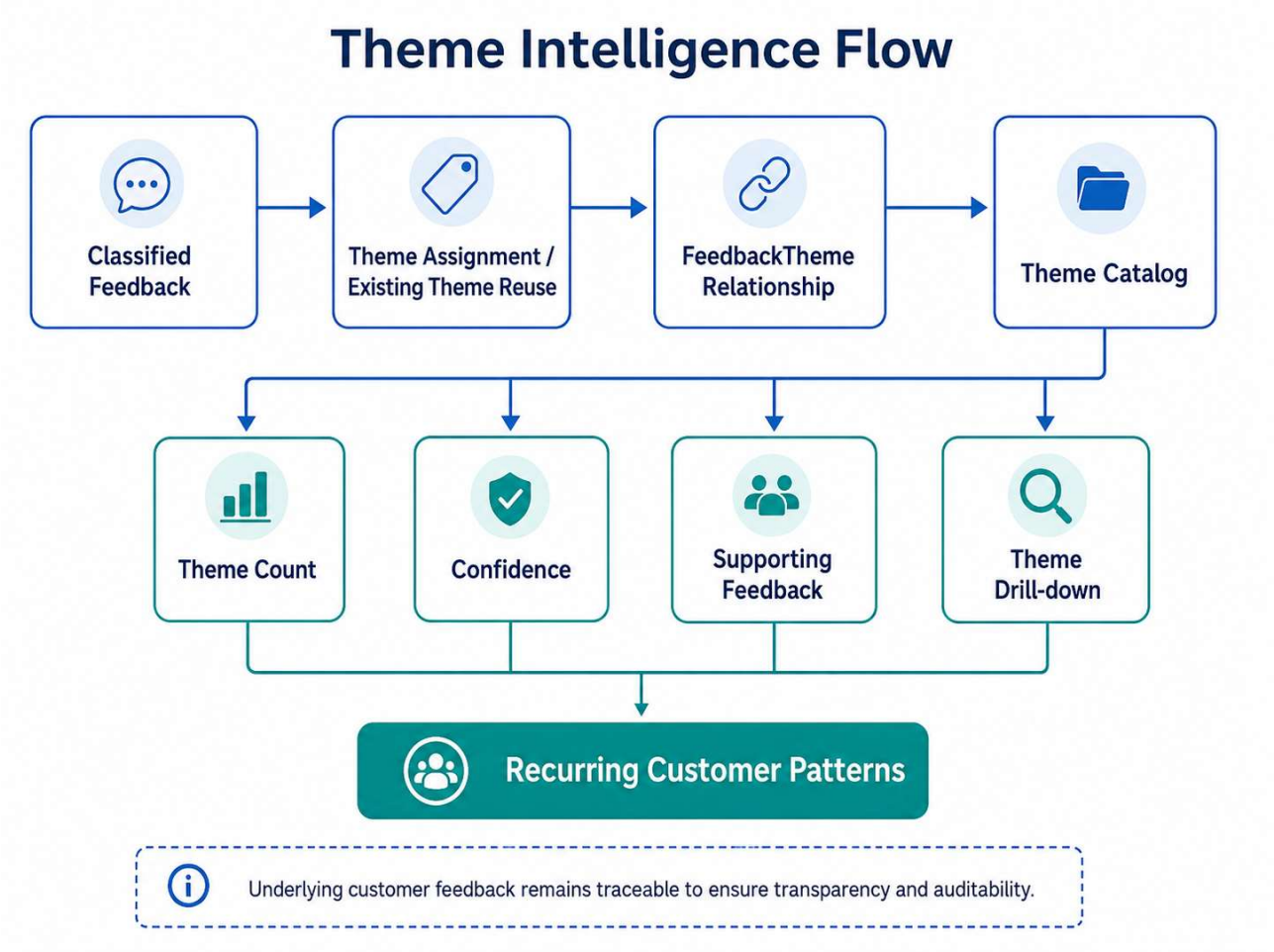
Validated results are stored in PostgreSQL.

This avoids repeatedly calling the AI provider whenever feedback is displayed.

19.6 Manual Reclassification

Authorized users can trigger reclassification when required.

Theme Intelligence



20.1 Purpose

Theme intelligence groups related feedback into recurring topics that can be analyzed at a higher level than individual comments.

20.2 Theme Assignment

The classification process can reuse existing workspace themes where appropriate.

New themes can be created when feedback does not fit existing categories.

20.3 Confidence

The feedback-theme relationship supports an assignment-confidence value.

20.4 Theme Catalog

Users can view:

- theme name;
- description;
- count;
- supporting feedback.

20.5 Evidence Drill-Down

A theme can be opened to inspect the feedback assigned to it.

This provides traceability from:

Theme

↓

Theme Count

↓

FeedbackTheme

↓

Underlying Customer Feedback

20.6 Result

Theme intelligence converts individual messages into recurring customer patterns while retaining access to the original evidence.

Trend Analysis

21.1 Purpose

A theme count alone does not reveal whether the topic is becoming more important.

The trends view therefore analyses theme activity over time.

21.2 Period Comparison

LOOP compares a selected current period against the immediately preceding period of equal duration.

Example:

Current:

1 August → 30 August

Previous:

2 July → 31 July

21.3 Daily Volume

Theme assignments are aggregated across dates to create daily volume data.

21.4 Spike Detection

The project's configured meaningful-spike rule uses:

Current count >= 3

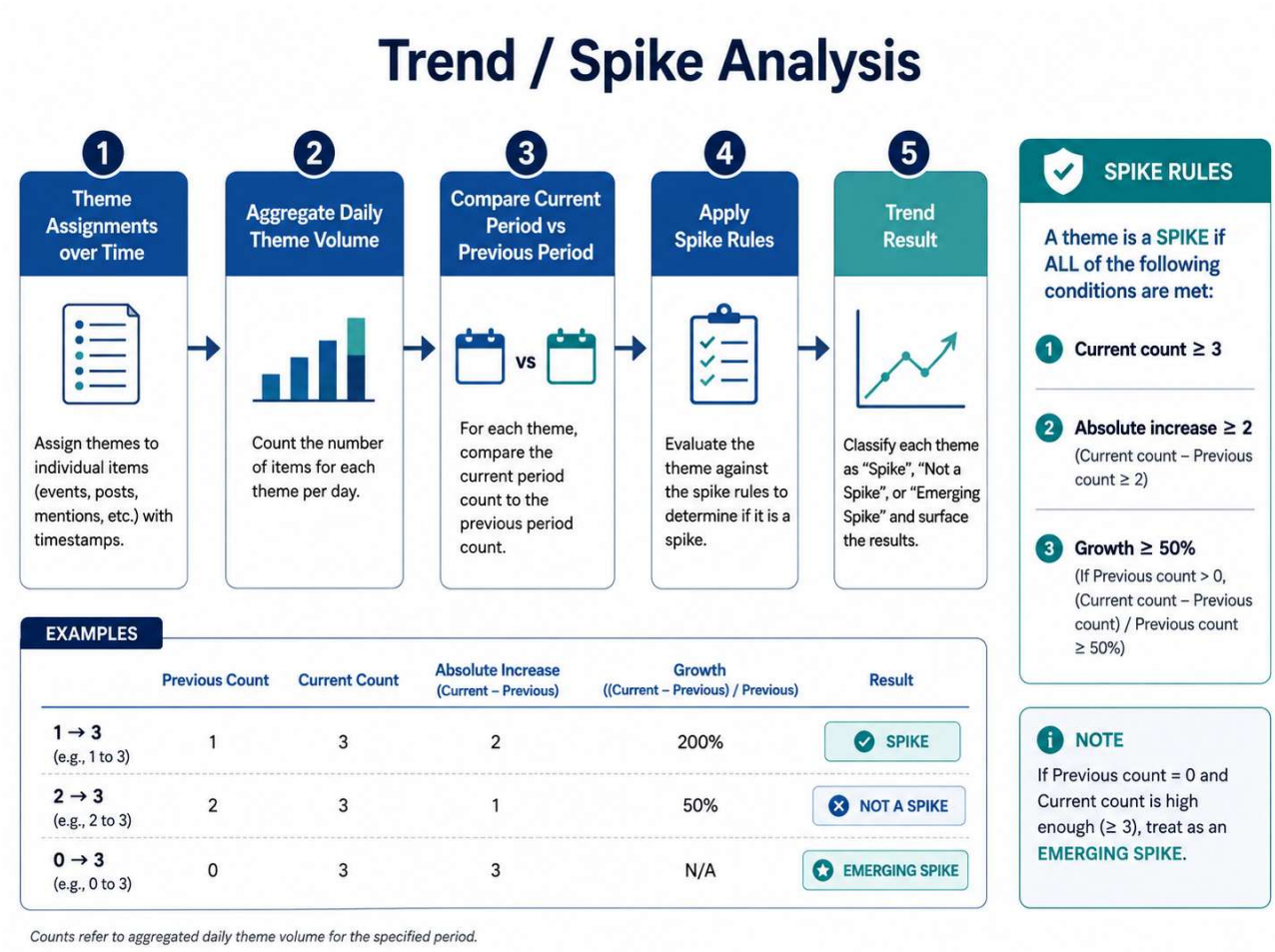
AND

Absolute increase >= 2

AND

Growth >= 50%

For a zero previous-period baseline, a current count meeting the configured evidence threshold may be treated as a newly emerging theme.



21.5 Example

Previous	Current	Outcome
1	3	Spike
2	3	Not a spike
2	4	Spike
0	2	Not a spike
0	3	Emerging spike
3	4	Not a spike
3	5	Spike

This deterministic calculation is application logic rather than an AI-generated judgment.

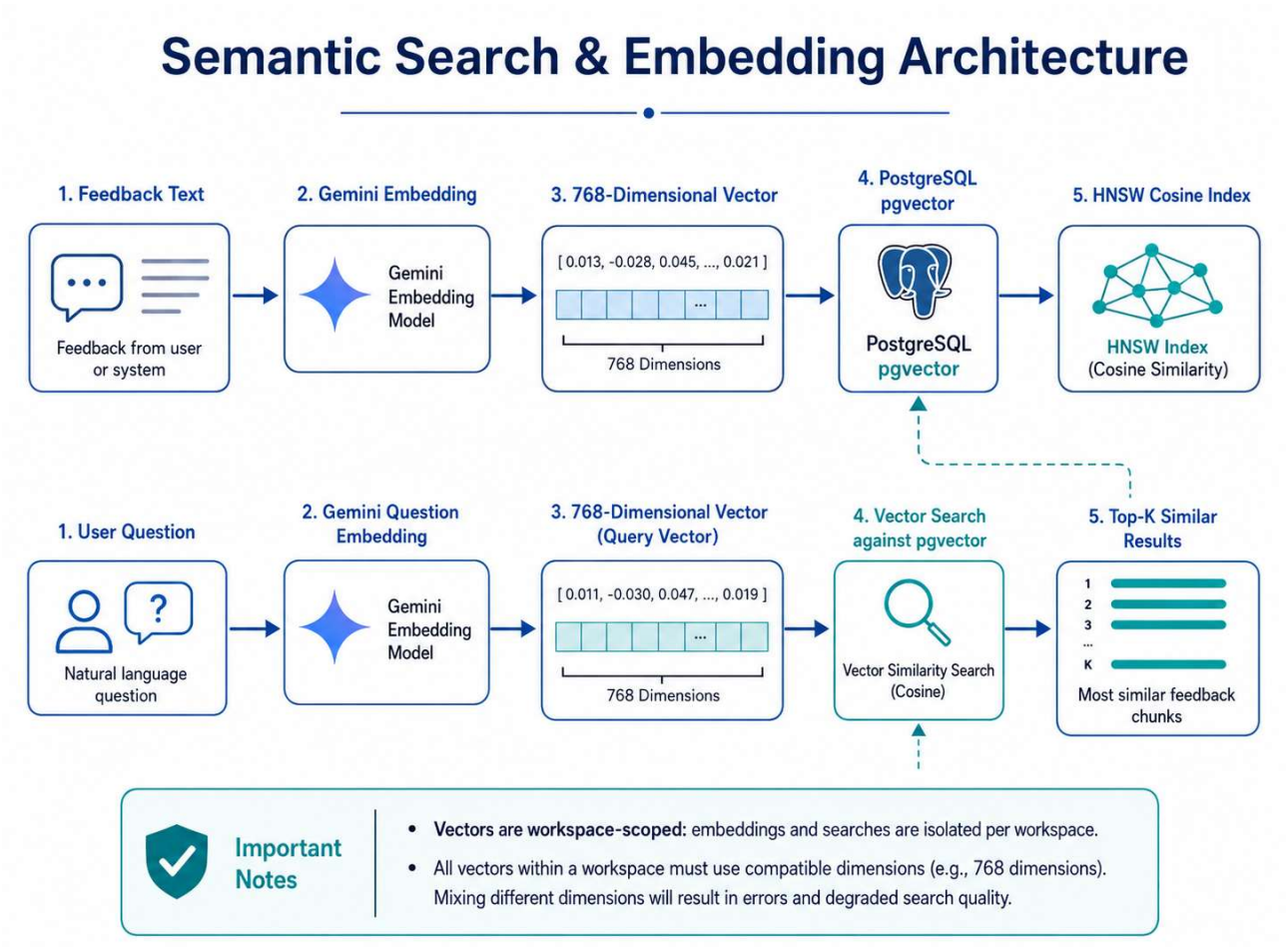
Semantic Search and Embeddings

22.1 Purpose

Keyword search is useful when the user knows the exact terms appearing in feedback.

Semantic search addresses a different problem: finding feedback that is conceptually related even when the exact words differ.

22.2 Embedding Pipeline



22.3 Vector Dimensions

The project uses:

768-dimensional embeddings.

All vectors used in the same search space must have compatible dimensions.

22.4 pgvector

pgvector extends PostgreSQL with vector storage and distance operations.

This allows LOOP to keep semantic vectors near the rest of the relational feedback data.

22.5 HNSW

The project uses an HNSW vector index with cosine-based similarity/distance search.

HNSW is designed to speed nearest-neighbor retrieval compared with comparing the question vector against every stored vector sequentially.

22.6 Tenant Isolation

Semantic retrieval is also workspace-scoped.

Embedding search must not retrieve vectors belonging to another tenant.

Ask LOOP / Retrieval-Augmented Generation

23.1 Purpose

Ask LOOP allows users to ask natural-language questions about customer feedback.

Example:

> “What are users saying about onboarding?”

23.2 Why Direct LLM Question Answering Is Insufficient

If the question were sent directly to a language model, the answer could be influenced by general model knowledge rather than the organization's customer feedback.

The application instead retrieves relevant internal evidence first.

23.3 RAG Pipeline

```
flowchart LR
    A[User Question] --> B[Gemini Question Embedding]
    B --> C[(pgvector)]
    C --> D[Top-K Relevant Feedback]
    D --> E[Evidence Context]
    E --> F[Gemini]
    F --> G[Structured Answer]
    G --> H[Citation Validation]
    H --> I[Grounded Answer + Evidence]
```

23.4 Retrieval Stage

The question is converted into an embedding.

The vector is compared with stored feedback embeddings.

Only results from the authenticated workspace are considered.

23.5 Generation Stage

The retrieved feedback items are supplied to Gemini.

The prompt instructs the model to work from the supplied evidence.

23.6 Evidence Validation

Returned feedback identifiers are checked against the actual retrieved result set.

An identifier that was not retrieved is not accepted as valid supporting evidence.

23.7 Insufficient Evidence

If relevant workspace feedback does not support a reliable answer, the system is designed to return an insufficient-evidence response instead of inventing feedback.

23.8 Security Consideration

Feedback itself is customer-controlled content.

The system therefore treats retrieved feedback as evidence/data rather than as trusted instructions for the AI model.

Voice-of-Customer Reports

24.1 Purpose

The report system creates a reusable summary of customer feedback for a selected date range.

24.2 Deterministic Calculations

Before Gemini generates narrative content, the application calculates actual statistics such as:

- total feedback;
- classification coverage;
- sentiment distribution;
- sentiment percentages;
- previous-period sentiment;
- sentiment shifts;
- top themes;
- representative evidence.

24.3 Report Pipeline

flowchart TD

A[Selected Report Period] --> B[Workspace Feedback Query]

B --> C[Deterministic Statistics]

C --> D[Top Themes]

C --> E[Sentiment Shifts]

C --> F[Representative Evidence]

D --> G[Gemini Narrative]

E --> G

F --> G

G --> H[Structured Validation]

H --> I[Evidence ID Validation]

I --> J[Persist Report]

24.4 Why Statistics Are Calculated First

A language model should not be relied upon to calculate factual business statistics if the application already has exact database values.

Therefore:

Database/Application Code → Facts

Gemini → Narrative

rather than:

Gemini → Facts + Narrative

24.5 Representative Feedback

Customer quotes/evidence displayed in a report are connected to stored feedback.

The model can reference feedback identifiers, which are validated before the original database content is displayed.

24.6 Saved Reports

Generated reports are persisted in PostgreSQL so they can be viewed later without regenerating the AI response.

24.7 Secure Sharing

Reports support public read-only capability links.

Share tokens are cryptographically generated, while only their SHA-256 hashes are stored in the database.

Links can be rotated or revoked.

Security

25.1 Security Goals

The project focuses on:

- protecting credentials;
- enforcing role permissions;
- protecting tenant boundaries;
- validating external input;
- keeping AI credentials private;
- protecting mutation routes;
- reducing disclosure through resource identifiers.

25.2 Password Hashing

User passwords are protected using scrypt hashes.

25.3 Authentication

NextAuth.js handles authenticated sessions.

25.4 Server-Side RBAC

Privileged API operations check the authenticated user's role on the server.

25.5 Workspace Isolation

Queries involving tenant-owned data include the authenticated user's workspace.

This prevents a client from switching tenants simply by altering a request parameter.

25.6 Zod Validation

Mutable API inputs are validated with Zod.

AI output is also validated before becoming trusted database state.

25.7 Gemini API Security

Gemini API calls are executed server-side.

The API key is stored in an environment variable:

GEMINI_API_KEY

It is not intentionally exposed through browser-facing NEXT_PUBLIC_* configuration.

25.8 Same-Origin Mutation Protection

The implementation includes same-origin checks for mutation operations to reduce cross-site request abuse.

25.9 Report Capability Tokens

Public report sharing uses:

1. cryptographically random token generation;
2. SHA-256 hashing;
3. database storage of the hash rather than raw token;
4. revocation and rotation capability.

25.10 Resource Disclosure

Unknown or cross-workspace resource identifiers are handled without intentionally revealing another tenant's data.

25.11 Repository Hygiene

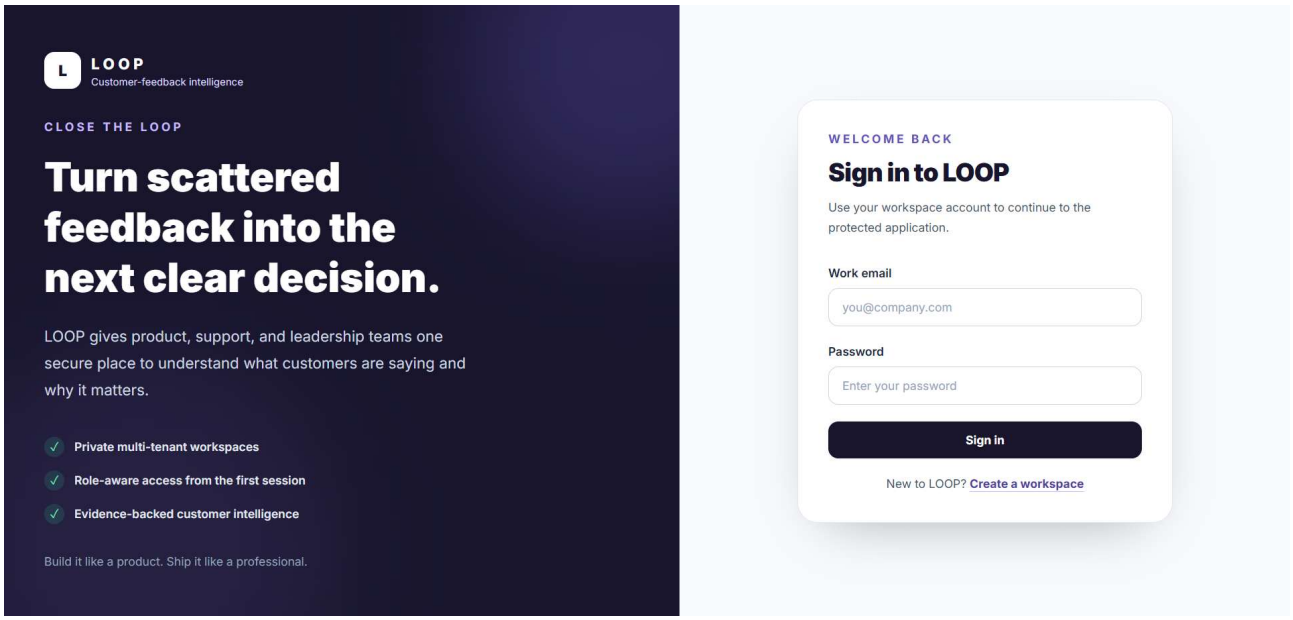
Sensitive files such as .env and secret configuration are excluded from Git.

Production secrets must be stored in deployment environment settings.

User Interface

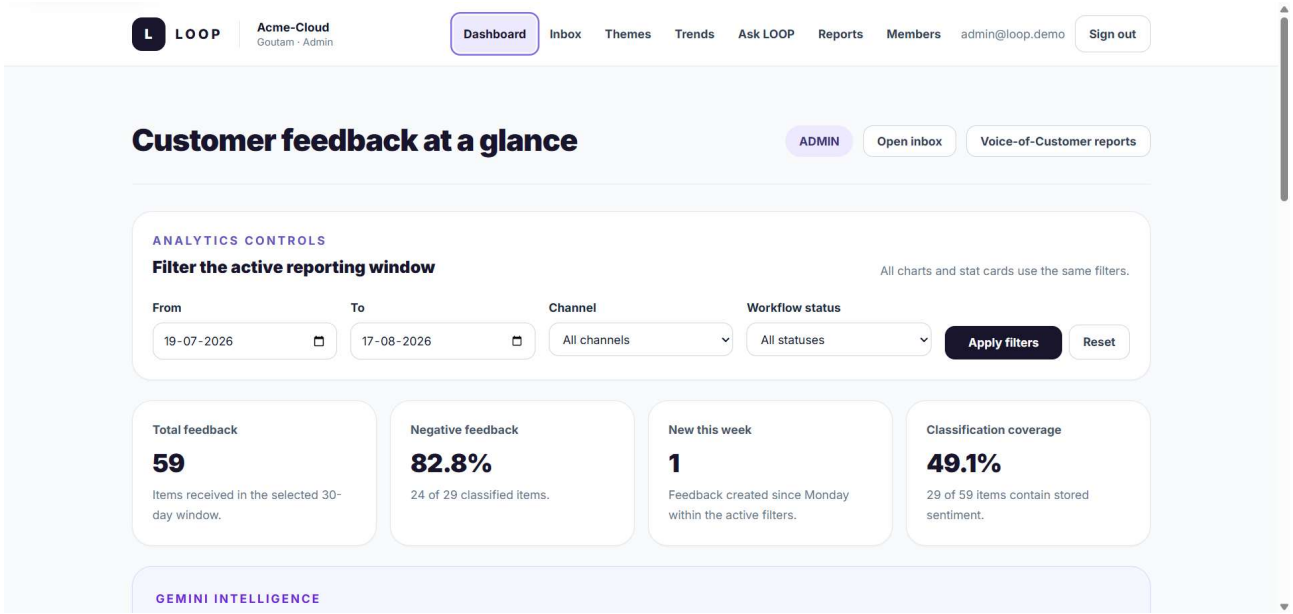
26.1 Login

The authentication interface provides access to the protected workspace application.



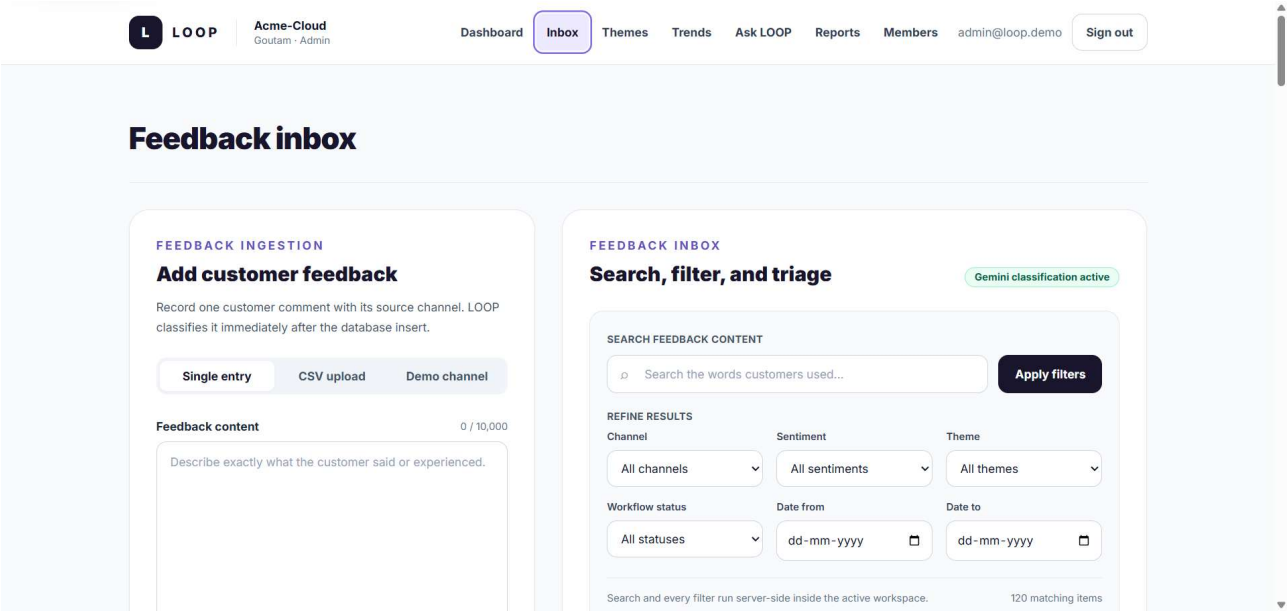
26.2 Dashboard

The dashboard summarizes feedback activity, sentiment, theme information, and persisted AI intelligence metrics.



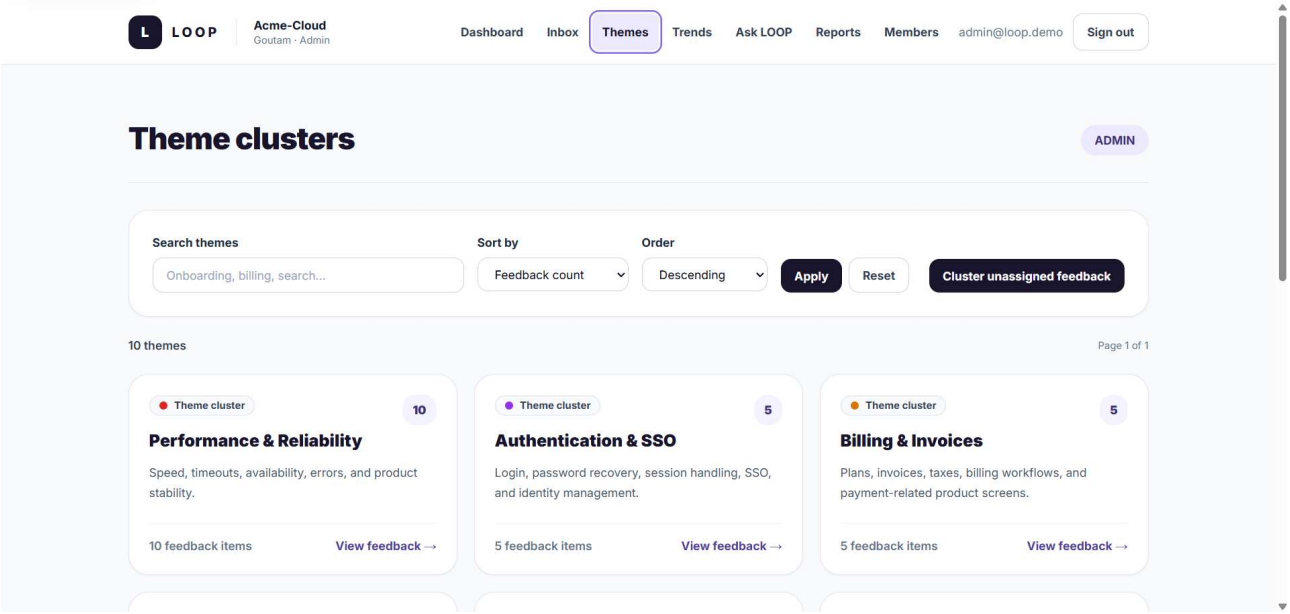
26.3 Feedback Inbox

The inbox provides search, filters, pagination, ingestion, classification information, and triage controls.



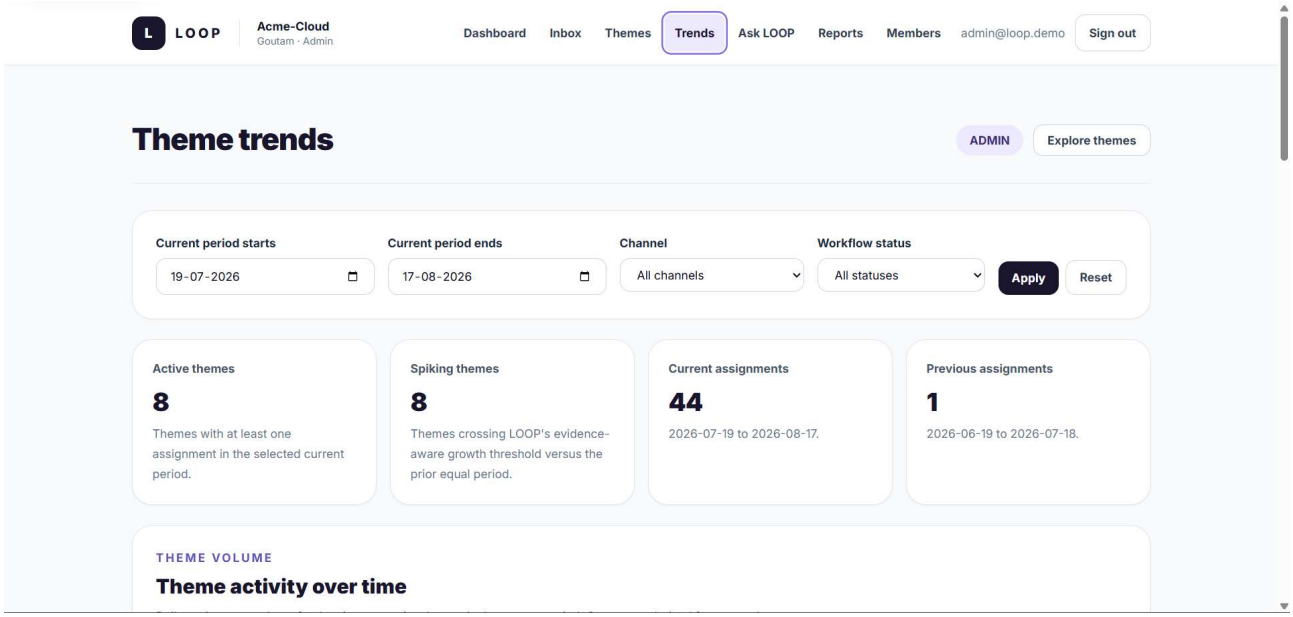
26.4 Themes

The Themes page shows recurring topics, counts, and supporting feedback.



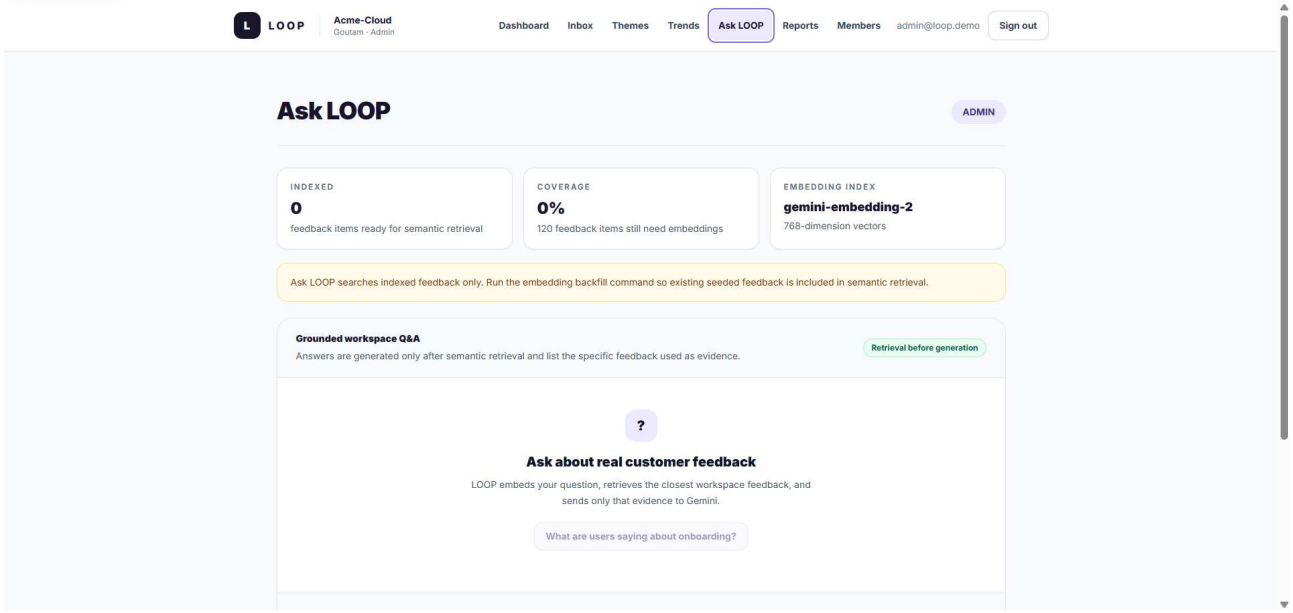
26.5 Trends

The Trends interface visualizes theme volume and period comparisons.



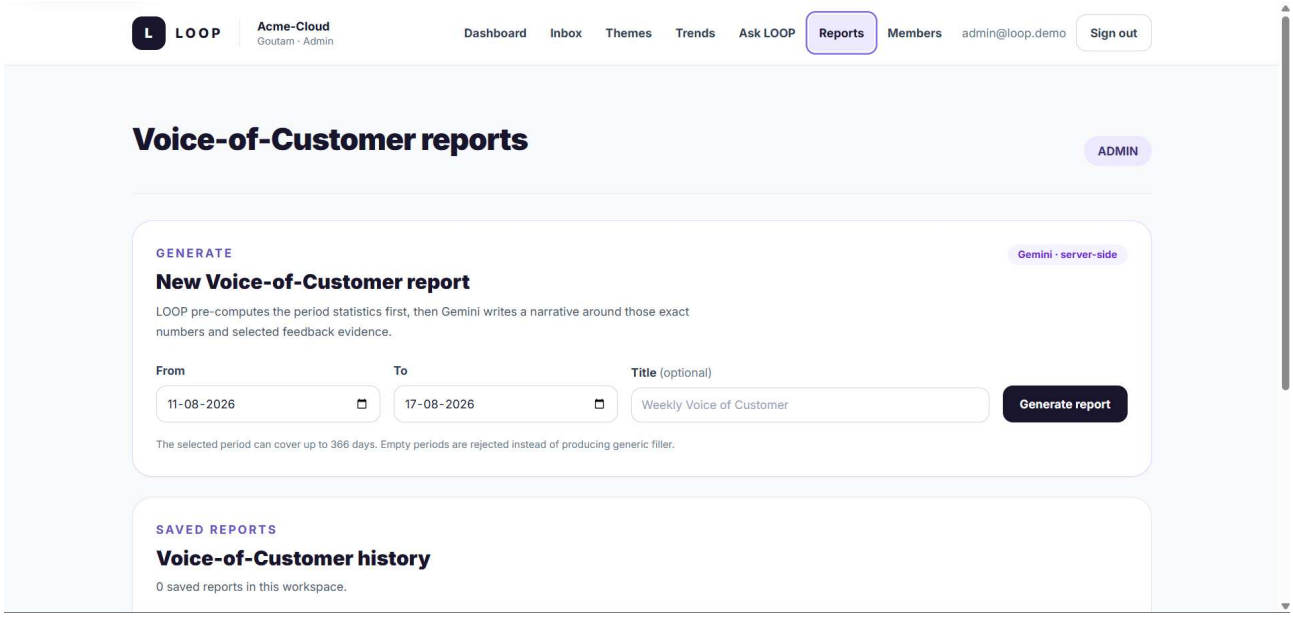
26.6 Ask LOOP

Ask LOOP provides retrieval-grounded natural-language feedback analysis.



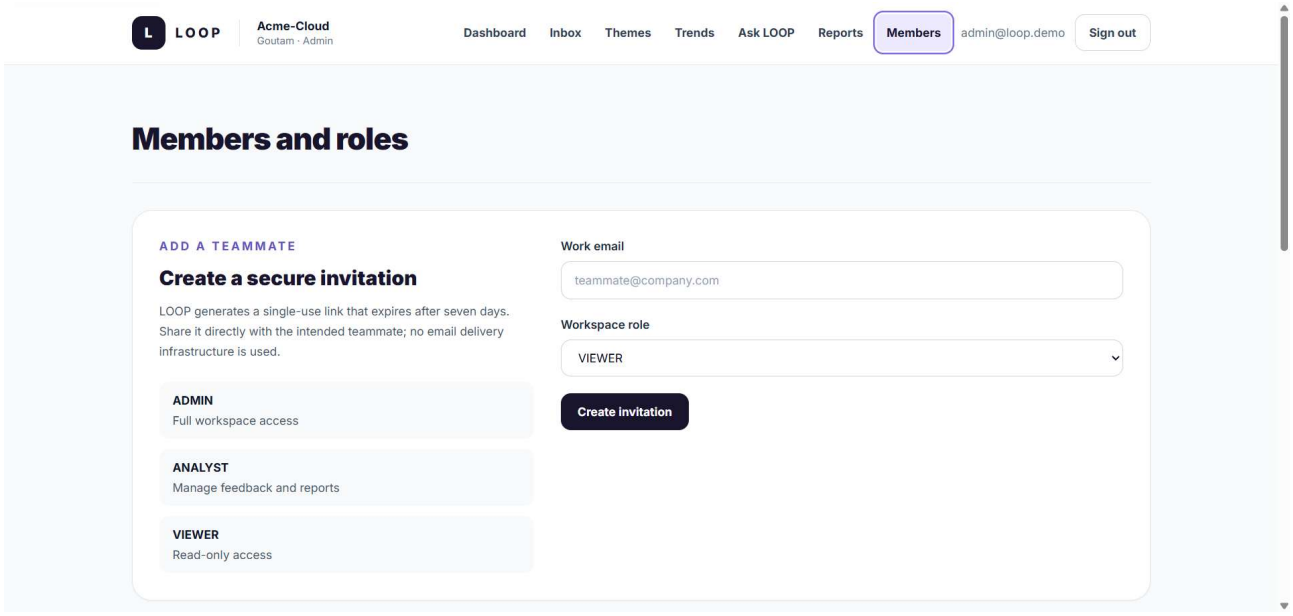
26.7 Voice-of-Customer Reports

The report interface displays generated summaries, themes, sentiment information, evidence, and recommended actions.



26.8 Admin Members

The Admin workspace interface provides member and role management.



26.9 UI Hardening

The final application work also included:

- loading states;
- empty states;
- route error states;
- 403 handling;
- 404 handling;
- responsive navigation;
- keyboard navigation considerations;
- visible focus behavior;
- reduced-motion handling.

Testing and Quality Assurance

27.1 Testing Approach

The available project information confirms a set of validation, build, repository, seed, AI-connectivity, and deployment-smoke workflows.

A comprehensive standalone unit/integration test suite is **not claimed**.

27.2 Type Checking

npm run typecheck

Checks TypeScript correctness.

27.3 Linting

npm run lint

Runs ESLint.

27.4 Formatting

npm run format:check

Verifies source formatting using the configured formatting workflow.

27.5 Production Build

npm run build

Builds the Next.js application after generating required Prisma code.

27.6 AI Verification

npm run ai:verify

Verifies Gemini structured-classification connectivity.

27.7 Seed Verification

npm run db:verify:seed

Checks final demo-data completeness.

The final demo dataset includes:

- one seeded workspace;
- Admin, Analyst and Viewer users;
- 120 feedback items;
- classifications;
- themes;
- semantic embeddings.

27.8 Repository QA

npm run qa:repo

Audits repository structure and final source/configuration hygiene.

27.9 Final QA

npm run final:qa

Runs the configured final verification chain, including:

- typecheck;
- lint;
- formatting check;
- build;
- seed verification;
- repository QA.
-

27.10 Production Smoke Test

npm run smoke -- --base-url=https://zidio-loop.vercel.app

The smoke workflow checks important deployed functionality including:

- landing/login availability;
- application health;
- dashboard access;
- core read APIs;
- member-management RBAC.

27.11 Submission Audit

npm run qa:submission

The strict audit is intended for final repository submission readiness.

27.12 Quality Commands

Command	Purpose
npm run typecheck	TypeScript verification
npm run lint	ESLint
npm run format:check	Formatting verification
npm run build	Production build
npm run ai:verify	Gemini connectivity check
npm run db:verify:seed	Final demo-data verification
npm run qa:repo	Repository QA
npm run final:qa	Combined quality workflow
npm run smoke -- --base-url=...	Deployment smoke testing
npm run qa:submission	Strict submission audit

No accuracy benchmark, performance benchmark, test coverage percentage, or uptime claim is made because those verified values were not supplied.

Deployment

28.1 Deployment Platform

The application is deployed using **Vercel**.

Live Application

https://zidio-loop.vercel.app/

28.2 Database

The production deployment requires PostgreSQL with pgvector support.

The exact hosted PostgreSQL provider used by the deployed project is:

Not specified in the available project information.

28.3 Migration Deployment

npm run db:migrate:deploy

applies committed Prisma migrations.

28.4 Final Demo Data

npm run db:seed:final

creates/prepares the final demo dataset and runs verification.

28.5 Deployment

The configured production deployment can be performed with Vercel.

npx vercel --prod

28.6 Smoke Testing

After deployment:

npm run smoke -- --base-url=https://zidio-loop.vercel.app

is used to check core production flows.

Results and Outcomes

The project demonstrates the successful integration of multiple application workflows.

28.1 Centralized Feedback

Feedback from manual entry, CSV imports, and simulated channels can be stored and viewed from one inbox.

28.2 Structured Classification

Customer text can be converted into structured:

- sentiment;
- score;
- themes;
- feature-area labels;
- rationale.

28.3 Theme Intelligence

Feedback can be aggregated by themes while preserving drill-down to the original evidence.

28.4 Trend Visibility

Theme volume can be analyzed across time and compared with a previous equivalent period.

28.5 Semantic Retrieval

Stored embeddings enable meaning-based feedback retrieval rather than keyword-only matching.

28.6 Retrieval-Grounded Q&A

Ask LOOP combines semantic retrieval with Gemini generation and supporting evidence.

28.7 Voice-of-Customer Reporting

The system combines deterministic statistics with generated narrative to produce saved customer-feedback reports.

28.8 Tenant Isolation

Workspace-owned operations are designed to remain scoped to the authenticated user's tenant.

28.9 Role-Specific Workflows

Admin, Analyst, and Viewer accounts have distinct permissions.

28.10 Demonstration Dataset

The final seeded demo workflow contains **120 feedback records** and three demo users across the three roles.

The project does not claim unverified model accuracy, performance, throughput, latency, or business-impact percentages.

Challenges and Solutions

29.1 Challenge 1 — Multi-Tenant Data Isolation

Problem

A tenant-aware application must prevent one workspace from accessing another workspace's feedback, themes, reports, embeddings, or members.

Solution

The backend derives workspace identity from the authenticated database user.

Tenant-owned queries are workspace-scoped.

Client-provided workspace identity is not considered authoritative.

Outcome

Tenant ownership becomes part of the normal server-side request path rather than depending on frontend behavior.

29.2 Challenge 2 — Reliable Structured AI Output

Problem

Language models can produce malformed JSON or values outside an application's expected schema.

Solution

Gemini is asked to return structured responses.

Responses are:

1. parsed;
2. validated with Zod;
3. persisted only after successful validation.

Controlled retry/failure handling is used for invalid results.

Outcome

AI output does not bypass normal application validation.

29.3 Challenge 3 — Grounding Ask LOOP

Problem

A model could generate a plausible answer based on general knowledge instead of workspace feedback.

Solution

LOOP uses retrieval before generation:

Question

→ Embedding

→ Vector Search

→ Relevant Feedback

→ Gemini

→ Grounded Answer

→ Evidence

Returned evidence IDs are checked against retrieved results.

Outcome

Users can see the customer feedback that supports an answer.

29.4 Challenge 4 — Embedding and Vector Retrieval

Problem

Semantic search requires compatible embedding dimensions and efficient nearest-neighbor retrieval.

Solution

The project uses:

- Gemini embeddings;
- 768-dimensional vectors;
- pgvector storage;
- cosine-based similarity;
- an HNSW index.

Outcome

Semantic feedback retrieval can operate directly within the PostgreSQL-backed application architecture.

29.5 Challenge 5 — Accurate VoC Reporting

Problem

Allowing the model to generate report statistics could produce unsupported numbers.

Solution

The application calculates factual report data before calling Gemini.

Only narrative interpretation is delegated to the model.

Outcome

The report separates deterministic numerical data from generated prose.

29.6 Challenge 6 — Production Deployment and QA

Problem

Local success does not guarantee deployment success.

Production requires correct:

- environment variables;

- authentication origins;
- migrations;
- seed data;
- database connectivity;
- external AI configuration.

Solution

The project includes:

- migration commands;
- final seed verification;
- build checks;
- repository QA;
- deployment smoke tests;
- final submission QA.

Outcome

Deployment verification is treated as a repeatable project workflow.

Limitations

The current project has several intentional or practical limitations.

30.1 Simulated Integrations

External feedback providers are simulated rather than connected to live third-party services.

30.2 No Billing

Subscription billing and payment processing are outside the implemented scope.

30.3 No Native Mobile Application

The current implementation is web-based.

30.4 No Real-Time Collaboration

Real-time collaborative features using WebSockets are outside the implemented scope.

30.5 No Email/SMS Infrastructure

Invitation or workflow delivery through full email/SMS infrastructure is not part of the current implementation.

30.6 AI Dependency

Classification, embeddings, Ask LOOP, and report narrative generation depend on availability and valid configuration of the Gemini service.

30.7 AI Output Remains Probabilistic

Validation can ensure output shape and retrieval grounding can improve evidence alignment, but these mechanisms should not be interpreted as guaranteeing AI correctness.

30.8 Test Coverage

The project includes several QA and smoke workflows, but a comprehensive dedicated unit/integration test suite has not been claimed in the available project information.

Future Scope

The following are future or stretch ideas and are **not current implemented features unless added after the source information used for this report**.

31.1 Saved Views

Allow users to save reusable inbox filters.

Example:

Negative onboarding feedback from the last 30 days

31.2 Suggested Actions

Convert recurring themes into a queue of draft product actions.

31.3 Sentiment Alerts

Notify or highlight meaningful negative-sentiment changes.

31.4 Real Third-Party Integrations

Possible future integration categories include:

- customer-support platforms;
- survey services;
- application-review platforms;
- customer-success systems.

Specific providers are **not specified in the available project information**.

31.5 Additional Enterprise Capabilities

The existing multi-tenant architecture could be extended with additional administration and workflow functionality.

Specific enterprise features should be defined only when required and are not claimed as part of the current implementation.

Conclusion

LOOP demonstrates a complete software-engineering workflow for transforming customer feedback into structured product intelligence.

The project combines conventional full-stack application requirements with AI capabilities rather than treating them as separate systems.

The implementation covers:

- authentication;
- RBAC;
- multi-tenancy;
- tenant isolation;
- relational database design;
- feedback ingestion;
- search;
- pagination;
- workflow management;
- analytics;
- AI classification;
- theme intelligence;
- trend analysis;
- semantic embeddings;
- vector search;
- retrieval-grounded question answering;
- Voice-of-Customer reports;
- secure sharing;

- deployment;
- quality verification.

An important technical lesson from LOOP is that incorporating an LLM into an application requires more than calling an API.

Generated output must be treated carefully.

Classification is validated before persistence. Ask LOOP retrieves workspace evidence before generation. Citations are validated against retrieved feedback. Report statistics are calculated using deterministic application logic before Gemini creates narrative text. AI API keys remain on the server.

Similarly, multi-tenancy requires more than storing a workspace field in the database. The application must derive tenant identity from the authenticated user and consistently scope tenant-owned operations.

As a Zidio Development Web Development Track internship project, LOOP demonstrates practical experience in full-stack development, SaaS-style architecture, database modelling, security, AI integration, RAG, vector search, analytics, deployment, and quality assurance.

Team

Team Member	Role / Contribution
Goutam Kushwah	Project Team Member
Gaurav Athode	Project Team Member
Sneha Sekar	Project Team Member
Mohan Sahu	Project Team Member

Individual contribution assignments were:

Not specified in the available project information.

Therefore, this report does not invent individual responsibilities.

References and Resources

34.1 Project Resources

1. **LOOP GitHub Repository**
   https://github.com/goutamkushwah/Project-loop
2. **LOOP Live Deployment**
   https://zidio-loop.vercel.app/

34.2 Technology Documentation Consulted During Project Development

The implementation uses technologies whose official documentation serves as the primary technical reference:

- Next.js documentation
- React documentation
- TypeScript documentation
- Tailwind CSS documentation
- PostgreSQL documentation
- Prisma documentation
- Zod documentation
- Google Gemini API documentation
- pgvector documentation
- Recharts documentation
- Vercel documentation

Specific documentation-page URLs or versions beyond the technologies listed above were not provided as formal references for this report.

Project Verification Notes

The following information could **not** be fully verified from the supplied project information and therefore has not been invented.

1. **Certificate format**
   The institution-specific certificate/declaration format, academic department, mentor name, guide name, signatures, dates, and seal requirements were not specified.
2. **Individual team responsibilities**
   Specific ownership of frontend, backend, AI, testing, documentation, or deployment work was not provided.
3. **Production PostgreSQL provider**
   PostgreSQL is confirmed, but the specific production database hosting provider was not specified.
4. **Hardware requirements**
   Formal minimum CPU, RAM, disk, or device specifications were not provided.
5. **Performance measurements**
   No verified latency, throughput, load, scalability, or response-time benchmarks were supplied.
6. **Availability measurements**
   No verified uptime or SLA figures were supplied.
7. **AI classification accuracy**
   No benchmark dataset or verified classification-accuracy percentage was supplied.
8. **Automated test coverage percentage**
   No code-coverage percentage was supplied.
9. **Comprehensive unit/integration test suite**
   The project contains type checking, linting, formatting verification, build verification, AI connectivity verification, seed verification, repository QA, final QA, and production smoke-test workflows. A comprehensive dedicated unit/integration test suite is not claimed.

10. **Exact individual Gemini generation-model identifier**

    The project information confirms Google Gemini through @google/genai and Gemini embeddings. This report does not rely on an exact generation-model identifier where one is not necessary.

11. **Live deployment availability at the exact time this report is read**

    The supplied production URL is:
    https://zidio-loop.vercel.app/

The URL is documented as the project's deployment location; continuous availability is not claimed.

End of Report

Project: LOOP — AI Customer Feedback Intelligence Platform

Internship: Zidio Development — Web Development Internship

Repository: https://github.com/goutamkushwah/Project-loop

Live Application: https://zidio-loop.vercel.app/